Mobile App Development & Java OOP — Question Bank with Answers

Part A: Mobile App Development

1. What is the Android operating system, and what type of OS architecture is it based on?

Android is an open-source, Linux-based operating system developed primarily by Google, designed mainly for touchscreen mobile devices such as smartphones and tablets. It is built on top of a modified Linux kernel and other open-source software, and is specifically designed for mobile devices with limited resources (memory, battery, processing power).

Architecture type: Android follows a layered (stack-based) software architecture, consisting of the Linux Kernel, Hardware Abstraction Layer (HAL), Native Libraries, Android Runtime (ART), Java API Framework, and System Apps. Each layer provides services to the layer above it.

2. What are the key features of the Android operating system, and how do they benefit developers and users?

- **Open Source**: Free to use and modify, allowing manufacturers and developers to customize it, reducing cost and increasing innovation.
- **Multi-tasking**: Supports running multiple applications simultaneously, improving user productivity.
- **Rich Development Environment**: Provides SDK, emulator, debugging tools, and libraries that speed up app development.
- **Connectivity Support**: Supports GSM, CDMA, Bluetooth, Wi-Fi, NFC, etc., enabling a wide range of communication features.
- **Storage Options**: SQLite database support allows structured data storage for apps.
- **Media Support**: Supports various audio, video, and image formats, helping build multimedia-rich apps.
- **Messaging and Web Browser**: Built-in SMS/MMS support and browser engine provide essential communication features out of the box.
- **Security**: Application sandboxing and permission-based access protect user data and device integrity.
- **GPS and Location Services**: Built-in location APIs enable location-based applications.

These features collectively reduce development effort, increase device compatibility, and provide users with a feature-rich, customizable experience.

3. Explain the Android architecture in detail, describing each layer and its role.

Android architecture consists of five main layers, from bottom to top.

Linux Kernel

The foundation layer that handles core system services: process management, memory management, security, network stack, and driver management (display, camera, Bluetooth, and so on). It acts as an abstraction layer between hardware and software.

Hardware Abstraction Layer (HAL)

Provides standard interfaces that expose device hardware capabilities to the higher-level Java API framework. It consists of multiple library modules, each implementing an interface for specific hardware components such as the camera or sensors.

Native Libraries and Android Runtime

The native C/C++ libraries include components like WebKit, SQLite, OpenGL, and SSL, used by various

system components. The Android Runtime (ART) executes DEX bytecode converted from Java or Kotlin bytecode, provides ahead-of-time and just-in-time compilation, handles garbage collection, and supports running multiple virtual machines on low-memory devices.

Java API Framework

Provides the entire feature set of Android through APIs written in Java, including the View System for UI components, Resource Manager, Notification Manager, Activity Manager, and Content Providers. This is the layer most app developers interact with directly.

System Apps

The top layer consists of built-in apps such as Phone, Contacts, Browser, and Email, along with user-installed apps. These use the APIs from the Java Framework layer.

4. What factors must be considered when developing a mobile application?

- **Device Fragmentation**: Different screen sizes, resolutions, OS versions, and hardware capabilities require responsive design and backward compatibility.
- **Performance**: Limited CPU and RAM compared to desktops require efficient algorithms, lazy loading, and memory optimization.
- **Battery Usage**: Background tasks, GPS, and network calls drain battery, so wake locks and background processing must be minimized.
- **Screen Sizes and Densities**: Apps must adapt their UI using responsive layouts such as ConstraintLayout and density-independent units like dp and sp.
- **Security**: User data must be protected through encryption, secure storage, permission management, and secure network communication using HTTPS.
- **Network Conditions**: Apps must handle offline mode, slow or intermittent connectivity, and data usage optimization.
- **User Experience**: A touch-friendly UI, intuitive navigation, and accessibility support are essential.
- **Platform Guidelines**: Following Material Design for Android or Human Interface Guidelines for iOS ensures consistency.
- **App Size**: An optimized APK or bundle size enables faster downloads and installs, especially in regions with limited data.
- **Testing Across Devices**: Functionality must be verified across various manufacturers, OS versions, and configurations.

5. What is the Android SDK, and what are its main components and tools?

The Android Software Development Kit (SDK) is a collection of tools, libraries, documentation, sample code, and APIs that developers use to build, test, and debug Android applications.

Main components and tools:

- **SDK Tools**: Includes the Android Emulator, ADB (Android Debug Bridge), and SDK Manager for managing platform versions and packages.
- **Build Tools**: Includes compilers and tools such as AAPT and the dex compiler, used to compile resources and generate APK or AAB files.
- **Platform Tools**: Includes ADB and Fastboot, used for communicating with connected Android devices and emulators.
- **Android Emulator**: Simulates Android devices on a computer for testing apps without physical hardware.
- **Platform APIs**: Java and Kotlin libraries for each Android version, providing access to system features for that API level.
- **System Images**: Pre-built OS images used by the emulator to run different Android versions.
- **Documentation and Samples**: Reference guides and sample projects that help developers learn the APIs.
- **Android Studio (IDE)**: Though technically separate, it bundles and integrates the SDK, providing a complete development environment with a code editor, layout designer, and debugger.

6. Define the following Android components with examples: Activity, Layout, Intent, Service, Content Provider, Broadcast Receiver.

Activity

A single, focused screen with which the user can interact, such as a login screen or a settings screen. Each activity is implemented as a subclass of the Activity class.

```
        public class LoginActivity extends AppCompatActivity { }
```

Layout

Defines the visual structure of the UI for an activity or fragment, specified in XML, describing how views such as buttons and text fields are arranged on the screen.

```
        <LinearLayout android:orientation="vertical">
        </LinearLayout>
```

Intent

A messaging object used to request an action from another app component, such as starting an activity or sending data. It can be explicit, targeting a specific component, or implicit, describing a general action.

```
        Intent intent = new Intent(this, ProfileActivity.class);
        startActivity(intent);
```

Service

A component that runs operations in the background without a user interface, even if the app is closed, such as playing music or downloading files.

```
        public class MusicService extends Service { }
```

Content Provider

Manages access to a structured set of app data, enabling data sharing between different applications, such as the Contacts provider.

```
        ContentResolver resolver = getContentResolver();
        Cursor cursor = resolver.query(ContactsContract.Contacts.CONTENT_URI, null, null, null, null);
```

Broadcast Receiver

A component that responds to system-wide broadcast announcements, such as battery low, screen on or off, or incoming SMS.

```
        public class BatteryReceiver extends BroadcastReceiver {
            public void onReceive(Context context, Intent intent) { }
        }
```

7. Describe the core components of an Android application and how they interact with each other.

Android applications are built using four core components.

Activities represent individual screens with a UI. They serve as the entry point for user interaction. Activities can start other activities using Intents and can pass data between them.

Services run in the background to perform long-running operations, such as network transactions or music playback, without a UI. Activities can start, stop, or bind to services to communicate with them.

Broadcast Receivers listen for and respond to system-wide broadcast messages from other apps or the system itself, such as “Wi-Fi connected” or “battery low.” They can trigger notifications or start other components.

Content Providers manage shared app data, allowing other apps with proper permissions to query or modify data through a standard interface.

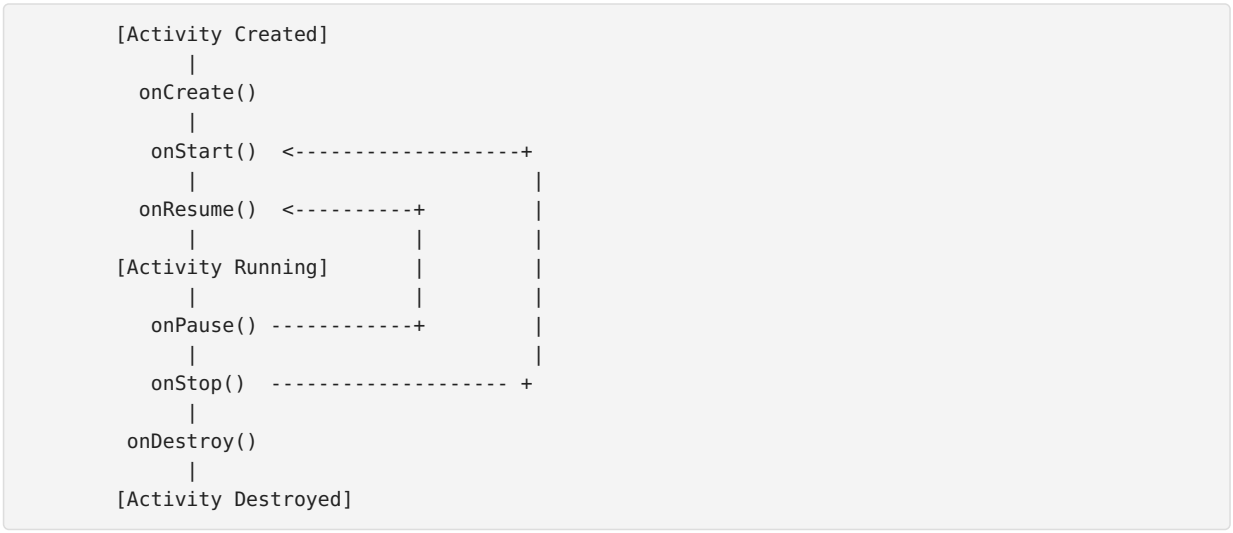
Interaction flow: An Activity can send an Intent to start another Activity or Service. A Service can send a Broadcast to notify Activities of completed tasks. A Broadcast Receiver can trigger a Service or display a notification when it receives specific system events. A Content Provider acts as a bridge enabling data exchange between unrelated apps, such as a Contacts app sharing data with a Messaging app.

All these components are declared in the AndroidManifest.xml file, which acts as the central configuration connecting them.

8. Explain the Activity lifecycle in detail with a diagram, and describe the types of Services and Broadcast Receivers along with their lifecycles.

Activity Lifecycle

The Activity lifecycle consists of seven main callback methods that the system calls as the activity moves through different states.



onCreate() is called when the activity is first created and is used for initialization such as setting up the UI and binding data.

onStart() is called when the activity becomes visible to the user.

onResume() is called when the activity starts interacting with the user and is in the foreground active state.

onPause() is called when the activity is partially obscured, for example when a dialog appears, and is used to pause ongoing actions.

onStop() is called when the activity is no longer visible, for example when another activity fully covers it.

onRestart() is called when the activity is restarting after being stopped, before onStart() is called again.

onDestroy() is called before the activity is destroyed, either by user action through finish() or by the system due to low memory.

Types of Services

Foreground Service: Performs operations noticeable to the user, must display a persistent notification, such as a music player or navigation app. It has higher priority and is less likely to be killed by the system.

Background Service: Performs operations not directly noticed by the user. Subject to background execution limits in newer Android versions, with Android 8.0 and above restricting background services heavily and encouraging the use of WorkManager.

Bound Service: Allows components such as Activities to bind to it using bindService(), enabling client-server interaction. The service runs as long as any component is bound to it and is destroyed when all

clients unbind.

Service lifecycle:

For a started service, the sequence is onCreate(), then onStartCommand(), then the running state, then onDestroy().

For a bound service, the sequence is onCreate(), then onBind(), then client interaction, then onUnbind(), then onDestroy().

Types of Broadcast Receivers

Static, manifest-declared receivers are registered in the AndroidManifest.xml. The system can wake up the app to deliver the broadcast even if the app is not running, although this is restricted in recent Android versions for most implicit broadcasts.

Dynamic, context-registered receivers are registered programmatically using registerReceiver() within an Activity or Service. They are active only while the registering component is alive and must be unregistered using unregisterReceiver().

Broadcast Receiver lifecycle: A BroadcastReceiver has a very short lifecycle. The onReceive() method is called when a matching broadcast is received, and the receiver object is considered dead once onReceive() returns, typically within ten seconds, or the system may kill it for being unresponsive.

9. Explain the structure and properties of the AndroidManifest.xml file, including its key elements and their purposes.

The AndroidManifest.xml is a mandatory configuration file located in the root of the app's project, providing essential information about the app to the Android build tools, the Android OS, and Google Play.

`<manifest>` is the root element, defining the package name and namespace for the app.

```
        <manifest xmlns:android="http://schemas.android.com/apk/res/android" package="com.example.myapp">
```

`<application>` contains declarations for app-level components and attributes such as icon, label, theme, and whether the app allows backup.

```
        <application android:icon="@mipmap/ic_launcher" android:label="@string/app_name"
android:theme="@style/AppTheme">
```

`<activity>` declares an Activity component. The intent-filter inside it specifies which intents the activity can respond to, and the MAIN action combined with the LAUNCHER category defines the app's entry point.

```
        <activity android:name=".MainActivity">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
```

`<service>` declares a Service component, `<receiver>` declares a Broadcast Receiver and the intents it listens for, and `<provider>` declares a Content Provider and its authority.

`<uses-permission>` declares permissions the app requires to function, such as internet access, camera, or location.

```
        <uses-permission android:name="android.permission.INTERNET" />
```

`<uses-feature>` declares hardware or software features the app requires, such as a camera, helping Google Play filter devices that lack required features.

`<uses-sdk>` specifies the minimum, target, and maximum SDK versions the app supports, although this is often moved to build.gradle in modern projects.

The Manifest essentially acts as a table of contents telling the Android system what components exist, what permissions are needed, what hardware is required, and what API level the app targets.

10. Differentiate between View and ViewGroup in Android, with examples of each.

Aspect	View	ViewGroup
Definition	A basic UI building block representing a single widget on screen	A container that holds and organizes other Views and ViewGroups
Purpose	Displays content or accepts user input	Manages layout and positioning of child views
Inheritance	Base class for all UI widgets	Subclass of View that acts as a container
Examples	TextView, Button, ImageView, EditText, CheckBox	LinearLayout, RelativeLayout, ConstraintLayout, FrameLayout
Children	Cannot contain other views	Can contain multiple child views or nested ViewGroups

Example:

```
<LinearLayout>
    <TextView android:text="Hello" />
    <Button android:text="Click Me" />
</LinearLayout>
```

Here, LinearLayout is a ViewGroup that arranges its child Views, the TextView and Button, vertically or horizontally.

11. Compare and describe the following layout types: LinearLayout, RelativeLayout, ConstraintLayout, FrameLayout.

LinearLayout arranges child views in a single direction, either horizontally or vertically, using the orientation attribute. It is simple to use for sequential UI elements but becomes inefficient with deeply nested layouts. It is best suited for forms with fields stacked vertically or toolbars with buttons arranged horizontally.

RelativeLayout positions child views relative to each other or to the parent container, using attributes like layout_below, layout_toRightOf, and layout_alignParentTop. It is more flexible than LinearLayout but can become complex and hard to maintain with many views. It is suited for UI where elements need to be positioned relative to specific anchors, such as a label next to an icon.

ConstraintLayout is a flexible layout that allows positioning and sizing of views using constraints relative to other views, the parent, or guidelines. It reduces nested view hierarchies, improves performance, and is the recommended layout for complex, responsive UIs, supporting chains, ratios, and bias for fine-tuned positioning. It is best for complex, responsive screens that must adapt across different screen sizes, which describes most modern Android apps.

FrameLayout is designed to display a single child view, or to stack multiple views on top of each other in z-order, with the last added view appearing on top. It is commonly used for fragment containers or overlay effects, such as displaying fragments, image overlays, or loading spinners over content.

Comparison summary:

Layout	Complexity	Performance	Flexibility	Best For
LinearLayout	Low	Good for simple cases	Limited	Simple sequential UI
RelativeLayout	Medium	Moderate	High	Relative positioning
ConstraintLayout	Medium to High	Best, flat hierarchy	Highest	Complex, responsive UI

FrameLayout	Low	Good	Low	Overlapping or single views

12. Explain Event Listeners in Android, including common types and how they are implemented.

Event Listeners are interfaces in the View class that contain callback methods, triggered when the registered event occurs on that view, such as a click or touch. They enable apps to respond to user interactions.

OnClickListener is called when the user clicks or taps a view.

```
        button.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                // Action on click
            }
        });
```

OnLongClickListener is called when the user performs a long press on a view.

```
        button.setOnLongClickListener(v -> {
            return true;
        });
```

OnTouchListener is called when a touch event such as down, move, or up is dispatched to the view, providing detailed motion information.

```
        view.setOnTouchListener((v, event) -> {
            return true;
        });
```

OnKeyListener is called when a hardware key event occurs while the view has focus.

OnFocusChangeListener is called when the focus state of a view changes, such as when an EditText gains or loses focus.

OnItemClickListener is used with AdapterViews such as ListView and GridView to handle clicks on individual items.

Implementation methods include using an anonymous inner class as shown above, using lambda expressions for functional interfaces in Java 8 and above, having the Activity or Fragment implement the listener interface directly, or using the android:onClick XML attribute, which calls a public method in the hosting Activity.

13. List and describe the important properties of the View class in Android, along with their purposes.

- **android:id**: Assigns a unique identifier to the view, used to reference it in code through findViewById().
- **android:layout_width and android:layout_height**: Define the size of the view, which can be match_parent, wrap_content, or a specific dimension.
- **android:padding**: Defines internal spacing between the view’s content and its border.
- **android:margin**: Defines external spacing between the view and its neighboring views or parent.
- **android:background**: Sets the background color, drawable, or image of the view.
- **android:visibility**: Controls whether the view is visible, invisible (takes up space but is not shown), or gone (removed from layout).
- **android:gravity**: Controls the alignment of content within the view, such as center, left, or right.
- **android:layout_gravity**: Controls how the view itself is positioned within its parent.
- **android:textSize and android:textColor**: For text-based views, control the size and color of displayed text.
- **android:onClick**: Binds a click event to a method defined in the hosting Activity.

- **android:enabled**: Determines whether the view can receive user interaction.
- **android:focusable and android:clickable**: Control whether a view can receive focus or respond to clicks.
- **android:alpha**: Controls the transparency of the view, from 0.0 to 1.0.
- **android:rotation, android:scaleX, android:scaleY, android:translationX/Y**: Control transformation properties for animations and effects.
- **android:tag**: Allows attaching arbitrary data or objects to a view for identification purposes.

These properties collectively control the appearance, behavior, positioning, and interactivity of UI elements.

14. Differentiate between Explicit and Implicit Intents in Android, with examples of each use case.

Explicit Intent specifies the exact component, or class, that should handle the intent by directly naming the target component using its class name. It is commonly used for navigation within the same application, where the developer knows exactly which Activity or Service to start.

```
        Intent intent = new Intent(CurrentActivity.this, ProfileActivity.class);
        intent.putExtra("user_id", 123);
        startActivity(intent);
```

This is used, for example, when navigating from a login screen to a home screen within the same app.

Implicit Intent does not specify a target component directly. Instead, it declares a general action to be performed, such as viewing a webpage, sharing content, or dialing a number, and the Android system determines which app or component can handle it based on registered intent filters. The system may show a chooser if multiple apps can handle the action.

```
        Intent intent = new Intent(Intent.ACTION_VIEW);
        intent.setData(Uri.parse("https://www.example.com"));
        startActivity(intent);
```

This is used, for example, when opening a webpage in a browser, sharing text using ACTION_SEND, dialing a phone number using ACTION_DIAL, or capturing a photo using ACTION_IMAGE_CAPTURE with whichever installed app the user chooses.

Aspect	Explicit Intent	Implicit Intent
Target	Specifies exact component (class)	Specifies action; system resolves target
Use Case	Internal app navigation	Inter-app communication
Flexibility	Less flexible, tightly coupled	More flexible, loosely coupled
Resolution	Direct, no ambiguity	Resolved via intent filters, may show chooser

Part B: Object-Oriented Programming in Java

15. What are the four main pillars of Object-Oriented Programming, and how is each one applied in Java with relevant examples?

The four main pillars of OOP are Encapsulation, Inheritance, Polymorphism, and Abstraction.

Encapsulation bundles data, or fields, and methods that operate on that data into a single unit, or class, while restricting direct access to internal data using access modifiers such as private and exposing controlled access through getters and setters.

```
        public class User {
            private String name;
            public String getName() { return name; }
```

```
    public void setName(String name) { this.name = name; }
}
```

Inheritance allows a class, the subclass, to acquire properties and behaviors, fields and methods, of another class, the superclass, using the extends keyword, promoting code reuse.

```
public class Animal {
    void eat() { System.out.println("Eating..."); }
}
public class Dog extends Animal {
    void bark() { System.out.println("Barking..."); }
}
```

Polymorphism allows objects to be treated as instances of their parent type, with the ability to take many forms. It is implemented through method overloading, which is compile-time polymorphism, and method overriding, which is runtime polymorphism.

```
class Animal {
    void sound() { System.out.println("Some sound"); }
}
class Cat extends Animal {
    @Override
    void sound() { System.out.println("Meow"); }
}
Animal a = new Cat();
a.sound(); // Outputs "Meow"
```

Abstraction hides implementation details and shows only essential features to the user, achieved using abstract classes and interfaces.

```
abstract class Shape {
    abstract double area();
}
class Circle extends Shape {
    double radius;
    double area() { return Math.PI * radius * radius; }
}
```

16. Explain the concepts of class and object in Java. How do constructors, instance variables, and methods work together to define an object's state and behavior?

Class: A class is a blueprint or template that defines the structure, or fields and variables, and behavior, or methods, that objects created from it will have. It does not occupy memory until an object is instantiated.

Object: An object is an instance of a class, a concrete entity created in memory based on the class blueprint, with its own values for the instance variables.

```
public class Car {
    String model;
    int speed;

    public Car(String model, int speed) {
        this.model = model;
        this.speed = speed;
    }

    public void accelerate() {
        speed += 10;
        System.out.println(model + " speed is now " + speed);
    }
}

Car myCar = new Car("Tesla", 50);
```

```
        myCar.accelerate();
```

How they work together:

Instance variables define the state of an object. Each object has its own copy of these variables, storing data unique to that instance, such as model and speed.

Constructors are special methods called when an object is created using new. They are used to initialize the instance variables to specific values at the time of object creation.

Methods define the behavior of an object, representing operations that can be performed on or by the object, often reading or modifying the instance variables, such as accelerate() modifying speed.

Together, a class defines what data an object will hold through instance variables, how that data is initialized through the constructor, and what actions can be performed through methods, combining to represent both the state and behavior of real-world entities in code.

17. Describe inheritance in Java, including types of inheritance supported, the use of the extends keyword, method overriding, and the role of the super keyword.

Inheritance is an OOP mechanism where a new class, the subclass or child class, derives fields and methods from an existing class, the superclass or parent class, promoting code reuse and establishing an “is-a” relationship.

The extends keyword is used to establish inheritance between classes. The subclass inherits all non-private members, fields and methods, of the superclass.

```
        class Vehicle {
            void start() { System.out.println("Vehicle starting"); }
        }
        class Car extends Vehicle {
            void drive() { System.out.println("Car driving"); }
        }
```

Types of inheritance supported in Java:

Single inheritance occurs when one class inherits from one superclass, such as Car extending Vehicle.

Multilevel inheritance is a chain of inheritance, where a class inherits from a class that itself inherits from another, such as SportsCar extending Car, which extends Vehicle.

Hierarchical inheritance occurs when multiple subclasses inherit from a single superclass, such as Car and Truck both extending Vehicle.

Multiple inheritance is not supported directly with classes in Java, to avoid ambiguity known as the diamond problem, but it is achievable through interfaces, where a class can implement multiple interfaces.

Method overriding occurs when a subclass provides its own specific implementation for a method already defined in its superclass. The method must have the same name, return type, and parameters. The @Override annotation is used for clarity and compile-time checking.

```
        class Vehicle {
            void start() { System.out.println("Generic start"); }
        }
        class Car extends Vehicle {
            @Override
            void start() { System.out.println("Car start with ignition"); }
        }
```

The super keyword is used within a subclass to refer to its immediate superclass and serves three purposes: calling the superclass constructor using super(args), which must be the first statement in the subclass constructor; accessing superclass methods using super.methodName(), useful when overriding a method but still wanting to execute the parent’s version; and accessing superclass fields using

super.fieldName, when a field is shadowed by a subclass field of the same name.

```
class Car extends Vehicle {
    Car() {
        super();
    }
    @Override
    void start() {
        super.start();
        System.out.println("Car-specific start logic");
    }
}
```

18. Explain polymorphism in Java, covering both compile-time and runtime polymorphism, and discuss its importance in Android development.

Polymorphism means “many forms,” the ability of an object, method, or reference variable to take multiple forms or behave differently based on context.

Compile-time polymorphism, or method overloading, occurs when multiple methods in the same class have the same name but different parameter lists, differing in number, type, or order of parameters. The compiler determines which method to call based on the method signature at compile time.

```
class Calculator {
    int add(int a, int b) { return a + b; }
    double add(double a, double b) { return a + b; }
    int add(int a, int b, int c) { return a + b + c; }
}
```

Runtime polymorphism, or method overriding, occurs when a subclass provides a specific implementation of a method already defined in its superclass. The decision of which method implementation to invoke is made at runtime, based on the actual object type rather than the reference type, a process called dynamic method dispatch.

```
class Animal {
    void makeSound() { System.out.println("Generic animal sound"); }
}
class Dog extends Animal {
    @Override
    void makeSound() { System.out.println("Bark"); }
}

Animal animal = new Dog();
animal.makeSound(); // Outputs "Bark"
```

Importance in Android development:

Android’s Activity, Fragment, and Service classes define lifecycle methods such as onCreate(), onResume(), and onPause() with default implementations. Developers override these methods in their own subclasses to add custom logic, relying entirely on runtime polymorphism, since the Android framework calls these methods on the actual object type, executing the overridden version.

```
public class MainActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }
}
```

Implementing interfaces like View.OnClickListener and overriding onClick() relies on polymorphism, allowing different views to have different click behaviors through the same interface method.

RecyclerView and ListView adapters override methods like onBindViewHolder() to provide custom binding logic for different view types, enabling flexible, reusable UI components.

Method overloading is frequently used in constructors of custom Views or utility classes, allowing flexible object creation with different parameter combinations, such as a custom View constructor overloaded to handle different initialization scenarios required by the Android View system.

19. Describe encapsulation and abstraction in Java, including access modifiers, abstract classes, and interfaces, and explain how these concepts are used in designing Android components.

Encapsulation is the practice of wrapping data, or fields, and methods that manipulate that data within a single unit, or class, and restricting direct access to the internal state from outside the class. This is achieved using access modifiers and getter and setter methods.

Access modifiers in Java:

A private member is accessible only within the same class. A default member, with no modifier, is accessible within the same package. A protected member is accessible within the same package and by subclasses, even in different packages. A public member is accessible from anywhere.

```
        public class Account {
            private double balance;

            public double getBalance() { return balance; }
            public void deposit(double amount) {
                if (amount > 0) balance += amount;
            }
        }
```

Encapsulation protects data integrity by preventing invalid states, such as a negative balance, and hides implementation details, allowing internal changes without affecting external code.

Abstraction is the concept of hiding complex implementation details and exposing only essential features or functionality to the user, achieved using abstract classes and interfaces.

Abstract classes are classes declared with the abstract keyword that cannot be instantiated directly and may contain both abstract methods, which have no body and must be implemented by subclasses, and concrete methods with implementation.

```
        abstract class PaymentMethod {
            abstract void processPayment(double amount);
            void logTransaction() { System.out.println("Transaction logged"); }
        }
        class CreditCardPayment extends PaymentMethod {
            @Override
            void processPayment(double amount) { System.out.println("Processing credit card payment: " +
amount); }
        }
```

Interfaces are completely abstract types, though in modern Java they can include default and static methods, that define a contract, a set of methods that implementing classes must provide. A class can implement multiple interfaces, achieving a form of multiple inheritance.

```
        interface Payable {
            void pay(double amount);
        }
        class Employee implements Payable {
            @Override
            public void pay(double amount) { System.out.println("Paying employee: " + amount); }
        }
```

Use in designing Android components:

For encapsulation, custom classes such as User, Repository, and ViewModel keep their fields private and expose data through getter methods, or in modern Android through LiveData and StateFlow getters, preventing external classes from directly modifying internal state and ensuring data consistency across the UI.

For abstraction with abstract classes, Android's Activity, Fragment, Service, and BroadcastReceiver are essentially abstract base classes. They provide a framework of lifecycle methods with some default behavior, while developers override specific methods such as onCreate() or onReceive() to implement custom logic, without needing to know the internal workings of the framework.

For abstraction with interfaces, contracts between layers are defined extensively, such as a Repository interface defining data operations like getUser() and saveUser(), while concrete implementations such as LocalRepository and RemoteRepository provide the actual logic. This decouples the UI layer from data source implementation details, supporting clean architecture and easier testing through mock implementations.

20. Discuss the importance of interfaces and abstract classes in Java for implementing callbacks, listeners, and reusable code structures commonly used in Android app development.

Interfaces for callbacks and listeners

Interfaces are fundamental to Android's event-driven architecture. A callback is a mechanism where a class passes a reference to one of its methods to another class, which then calls back that method when a specific event occurs.

Android's View class defines listener interfaces like OnClickListener, OnLongClickListener, and OnTouchListener. When a user interacts with a UI element, the system calls the corresponding interface method on the registered listener object.

```
button.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        // Respond to click event
    }
});
```

Developers also define their own custom callback interfaces to enable communication between components, such as Fragment-to-Activity communication or asynchronous task completion notifications, without creating tight coupling between classes.

```
interface OnDataLoadedListener {
    void onSuccess(String data);
    void onError(String errorMessage);
}

class DataLoader {
    private OnDataLoadedListener listener;
    public void setListener(OnDataLoadedListener listener) { this.listener = listener; }

    public void loadData() {
        if (success) listener.onSuccess(data);
        else listener.onError("Failed to load");
    }
}
```

Importance: Decoupling is achieved because the class triggering the callback doesn't need to know the specific implementation details of the listener, only the interface contract, allowing different classes to provide different implementations. A common Android pattern is Fragment-Activity communication, where a Fragment defines a listener interface that the hosting Activity implements, allowing the Fragment to send events or data back to the Activity without directly referencing it.

Abstract classes for reusable code structures

Abstract classes are used when multiple related classes share common code or state but also require some methods to be implemented differently by each subclass.

RecyclerView.Adapter is an abstract class that provides the framework for binding data to views, requiring subclasses to implement onCreateViewHolder(), onBindViewHolder(), and getItemCount(), while providing common adapter functionality. AsyncTask, in legacy code, is an abstract class requiring implementation of doInBackground() while providing the threading framework. Developers often create a BaseActivity abstract class containing common functionality, such as setting up a toolbar or handling common navigation, which all other activities extend, reducing code duplication.

Importance: Code reuse is achieved because common logic is implemented once in the abstract class and shared by all subclasses. The template method pattern is supported, where an abstract class defines a skeleton of an algorithm, such as the loading sequence for an adapter, with specific steps implemented by subclasses, ensuring consistent structure while allowing customization. Partial implementation is possible, since, unlike interfaces in older Java versions, abstract classes can provide both implemented and unimplemented methods, useful when some default behavior should be shared but other parts must be customized.

Interfaces versus abstract classes: Interfaces are best for defining capabilities or contracts that unrelated classes can implement, such as listeners or repository contracts, supporting multiple implementation. Abstract classes are best for related classes sharing common state and partial implementation, such as base Activities or Adapters, supporting single inheritance with shared code.